

Импорт важных библиотек

```
import numpy as np # библиотека Numpy для операций линейной алгебры и прочего
import pandas as pd # Библиотека Pandas для работы с табличными данными

import matplotlib.pyplot as plt # библиотека Matplotlib для визуализации
import seaborn as sns # библиотека Seaborn для визуализации данных из Pandas

from sklearn.datasets import fetch_openml # функция библиотеки scikit-learn для загрузки данных с сайта OpenML

from sklearn.decomposition import PCA # Метод главных компонент от мира scikit-learn

import tqdm

from sklearn.preprocessing import StandardScaler # Стандартизация от мира scikit-learn

import plotly.express as px # yet another 3-D рисовалка
from mpl_toolkits.mplot3d import Axes3D

plt.style.use('dark_background')
```

Набор данных MNIST

В качестве данных мы рассмотрим набор данных MNIST

Классический набор изображений рукописных цифр

Размер изображений 28 на 28, что можно рассматривать как большую таблицу признаков из 784 столбцов

Единственный канал - "оттенки серого"

Изображения хранятся в 8-битном формате, т.е. оттенков серого 256 от 0 (черный) до 255 (белый)

```
# воспользуемся функцией fetch_openml для загрузки данных с сайта OpenML
# для этого нам необходимо знать имя набора данных и версию
# также указываем что хотим чтобы скачалось в формате признаки/целевые метки (return_X_y)
# иначе он выгрузит в формате словаря с полной выкладкой
# наконец попросим выгрузить данные в формате dataframe (as_frame)
# иначе будут numpy массивы
X, y = fetch_openml('mnist_784',
                    version=1,
                    return_X_y=True,
                    as_frame = True,
                    parser='auto')
X = X / 255. # нормализуем данные - из диапазона от 0 до 255 в диапазон от 0 до 1
```

Разные примеры

```
#@title Разные примеры { run: "auto" }
n = 15258 #@param {type:"slider", min:1, max:69984, step:1}

# создаем полотно
fig = plt.figure( figsize=(15,8) )
# на котором визуализируем 10 примеров изображений
# начиная с индекса n
for i in range(0,10):
    # каждое изображение будет на своем сабплоте
    ax = fig.add_subplot(2,5,i+1)
    # пишем какое число должно быть
    ax.set_title("Число: {}".format(str(y[i+n])) , size=35)
    # отрисовываем с помощью функции matshow
    # для этого нам вектор-строку из 784 элементов
    # нужно преобразовать в матрицу 28 на 28
    ax.matshow(1-X.loc[i+n, 'pixel1': 'pixel784'].values.reshape((28,28)).astype(float),
               cmap='Greys')
    plt.setp(ax.get_xticklabels(), visible=False)
    plt.setp(ax.get_yticklabels(), visible=False)
    plt.tight_layout()
```

n

15258

[Скрыть код](#)



С помощью слайдера можно выбрать другие изображения для визуализации

✓ Применяем Метод Главных Компонент к Данным MNIST

И вновь стандартная для нас последовательность

- Создаем модель-объект (в этот раз ограничимся, допустим 100 компонентами)
- Обучаем через `.fit()`

```
pca = PCA(n_components = 784)
pca.fit(X)
```

PCA
PCA(n_components=784)

Оцениваем пространство главных компонент

```
pca_result_mnist = pca.transform(X)
```

✓ Визуализация

Разная

```
EV=pca.explained_variance_ratio_
```

✓ Восстановление данных

Сравним восстановленные изображения с исходными

```
##@title { run: "auto" }
##@markdown ### Разные примеры
n = 36523 ##@param {type:"slider", min:1, max:69984, step:1}
##@markdown ### Количество компонент для восстановления
n_comp = 10 ##@param {type:"slider", min:0, max:100, step:1}
```

Разные примеры

n

36523



```
pca_inverse = np.dot(pca_result_mnist[:, :n_comp], pca.components_[:n_comp])

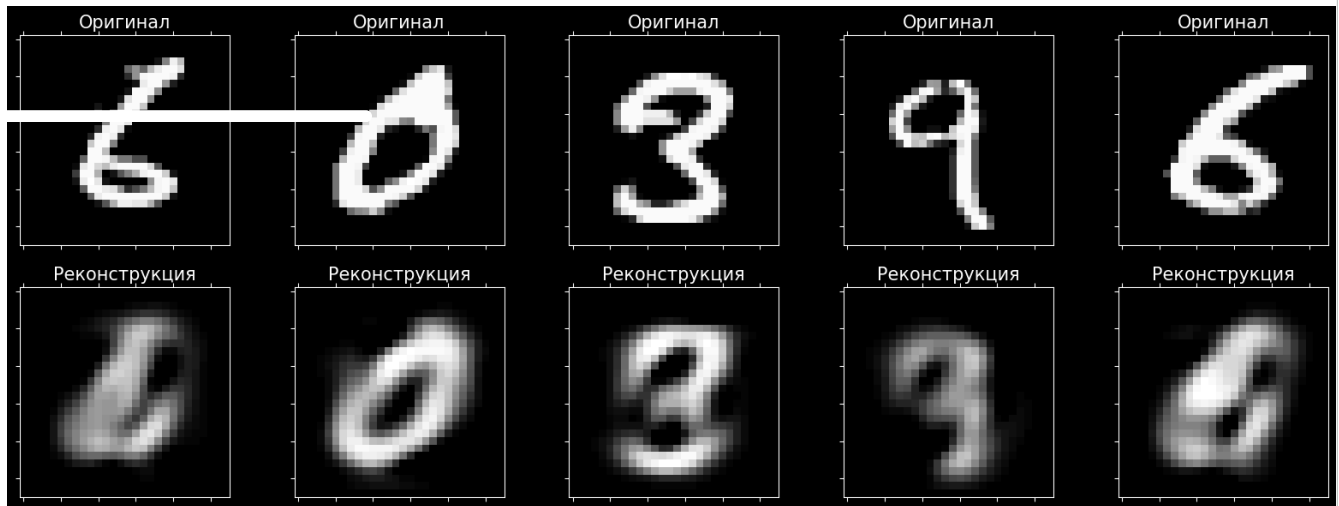
fig=plt.figure(figsize=(20,7))
for i in range(0,5):
    ax = fig.add_subplot(2,5,i+6)
    ax.set_title("Реконструкция", size=15)
    ax.imshow(1-pca_inverse[i+n,:].reshape((28,28)).astype(float),cmap=
    plt.setp(ax.get_xticklabels(), visible=False)
    plt.setp(ax.get_yticklabels(), visible=False)

for i in range(0,5):
    ax = fig.add_subplot(2,5,i+1)
    ax.set_title("Оригинал", size=15)
    ax.imshow(1-X.loc[i+n,'pixel1':'pixel784'].values.reshape((28,28))
    plt.setp(ax.get_xticklabels(), visible=False)
    plt.setp(ax.get_yticklabels(), visible=False)
```

Количество компонент для восстановления

n_comp

10

[Скрыть код](#)

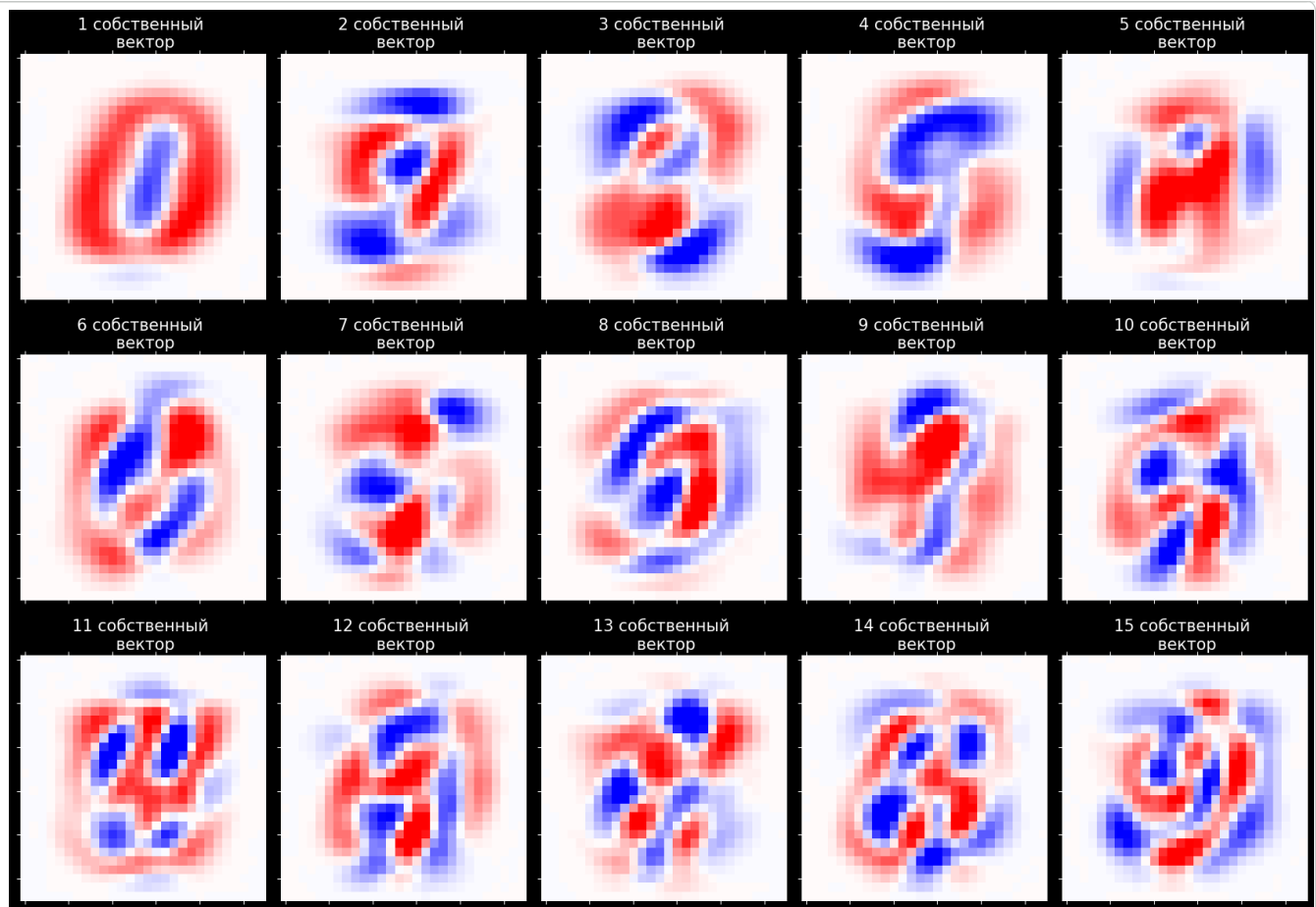
▼ Попытка понять что значат главные компоненты

Визуализируем собственные вектора сделав преобразование от вектор-строки из 784 элементов в матрицу 28 на 28

Поскольку, напоминаю, что отдельный собственный вектор это веса всех исходных признаков в отдельных главных компонентах.

Т.е. это вектор соразмерный с исходными данными, а значит его тоже можно представить в виде изображения

```
W_pca=pca.components_
fig=plt.figure(figsize=(17,12))
for i in range(0,15):
    ax = fig.add_subplot(3,5,i+1,title=str(i+1) )
    ax.set_title("{} собственный \n вектор".format(str(i+1)), size=15)
    ax.imshow(W_pca[i].reshape((28,28)).astype(float), cmap = 'bwr', vmin=-0.1,vmax = 0.1)
    plt.setp(ax.get_xticklabels(), visible=False)
    plt.setp(ax.get_yticklabels(), visible=False)
plt.tight_layout();
```



✓ Огонь визуализация как по мне

Взято [отсюда](#) (первоисточник может не он, но тем не менее)

```
from matplotlib.offsetbox import OffsetImage, AnnotationBbox

def plot_decomposition(image, components, coef, cmap='bwr'):
    image_shape = image.shape
    plt.figure(figsize=(20, 3))
    ax = plt.gca()

    imagebox = OffsetImage(image, zoom=5, cmap="gray")
    ab = AnnotationBbox(imagebox, (.05, 0.4), pad=0.0, xycoords='data')
    ax.add_artist(ab)
    sorting = np.argsort(np.abs(coef))[:, -1]

    for i in range(4):
        imagebox = OffsetImage(components[sorting[i]].reshape(image_shape), zoom=5., cmap=cmap)

        ab = AnnotationBbox(imagebox, (.3 + .2 * i, 0.4),
                            pad=0.0,
                            xycoords='data')
        ax.add_artist(ab)
        if i == 0:
            plt.text(.18, .3, r'{:.2f}'.format(coef[sorting[i]]), fontdict={'fontsize': 20})
        else:
            plt.text(.165 + .202 * i, .3, r'+ {:.2f}'.format(coef[sorting[i]]), fontdict={'fontsize': 20})
```



```
plt.text(.96, .25, '+ ...', fontdict={'fontsize': 20})

plt.text(.13, .3, '~=', fontdict={'fontsize': 20})

plt.axis("off")
```

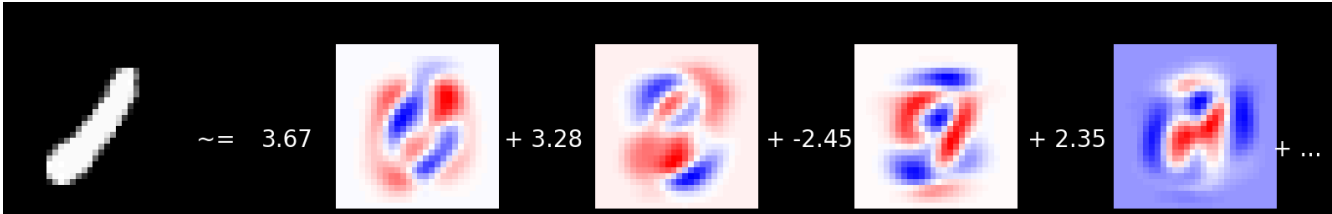
Визуализируем 4 главные компоненты с наибольшими весами для конкретного примера

Разные примеры

n

10257

[Показать код](#)



3Д Графоний

Складываем результаты PCA в Датафрейм Pandas

Не забываем про столбец меток цифр

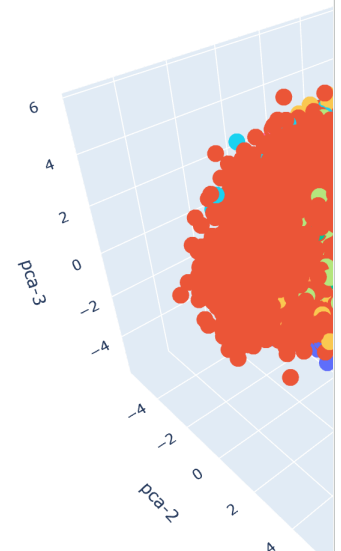
```
pca_names = ['pca-'+ str(x+1)for x in range(784)]
df = pd.DataFrame(pca_result_mnist, columns = pca_names)
df['label'] = y
df
```

	pca-1	pca-2	pca-3	pca-4	pca-5	pca-6	pca-7	pca-8	pca-9	pca-10	...	pca-776	pca-777	pca-778	pca-779	pca-780
0	0.479432	-1.240133	-0.200517	-2.183914	-0.192981	-0.851252	0.916523	0.740486	-1.407055	0.798216	...	0.0	0.0	0.0	0.0	-6.003989
1	3.962722	-1.137112	2.259297	-1.902288	-3.299915	-0.570459	-0.189543	-0.400906	0.136417	0.593196	...	0.0	0.0	0.0	0.0	2.207919
2	-0.231357	1.543912	-0.635287	2.075376	-1.228076	0.035177	-3.376432	1.467802	-0.307848	-0.755756	...	0.0	0.0	0.0	0.0	-1.485812
3	-3.125353	-2.382044	1.158913	0.427893	0.100652	2.736703	-1.860816	-0.391280	1.170990	0.017776	...	0.0	0.0	0.0	0.0	-4.100596
4	-1.534584	2.861977	0.054537	-0.952526	0.387345	0.140886	-0.342973	-1.630503	0.785482	0.416501	...	0.0	0.0	0.0	0.0	-1.098180
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
69995	1.198800	-2.155471	-0.087556	1.089978	3.328259	3.629437	3.067318	1.082658	-2.142159	-0.115546	...	0.0	0.0	0.0	0.0	-1.948983
69996	2.232102	-2.741283	-3.271962	-0.623487	0.369768	-0.750803	2.637972	3.074762	-0.190614	-0.150009	...	0.0	0.0	0.0	0.0	2.151621

Для поворота рисунков мы используем библиотеку plotly.express

Больше интерактива богу интерактивов

```
fig = px.scatter_3d(df, x=df["pca-1"], y=df["pca-2"], z=df["pca-3"],
                    color=df["label"])
fig.update_layout(margin=dict(l=0, r=0, b=0, t=0))
```



Давайте немного покласифицируем

```
from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_validate
from sklearn.metrics import make_scorer

from sklearn.metrics import confusion_matrix

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

Функция чтобы посчитал кросс-валидацию

```
def cross_validation(X, y, model, scoring, cv_rule):
    scores = cross_validate(model, X, y,
                           scoring=scoring, cv=cv_rule)
    print('Ошибка на кросс-валидации')
    DF_poly = pd.DataFrame(scores)
    display(DF_poly)
    print('\n')
    print(DF_poly.mean()[2:])
```

Функция чтобы нарисовать значимость признаков

```
def feature_importance_plotter(model, features_names):
    feature_importance = model.feature_importances_

    sorted = np.argsort(feature_importance)

    ypos = np.arange(len(features_names))

    fig = plt.figure(figsize=(8,4))
    plt.barh(ypos, feature_importance[sorted])
    plt.xlim([0,1])
    plt.ylabel('Параметры')
    plt.xlabel('Значимость')
    plt.yticks(ypos, features_names[sorted])

    scoring = {'accuracy': make_scorer(accuracy_score),
               'precision': make_scorer(precision_score, average = 'macro'),
               'recall': make_scorer(recall_score, average = 'macro'),
               'f1_score': make_scorer(f1_score, average = 'macro')}

    cv_clf = StratifiedKFold(n_splits=5, shuffle = True, random_state=42)
```



```
X = df.drop(columns = ["label"]).values
y = df["label"].values
features_names = df.drop(columns = ["label"]).columns

n = 15

features_names = features_names[0:n]
X_train, X_val, y_train, y_val = train_test_split(X[:,0:n], y, test_size=0.1, random_state=42)
```

Случайный Лес

```
from sklearn.ensemble import RandomForestClassifier
```

ГиперПараметры Случайного Леса

```
#@title ГиперПараметры Случайного Леса{ run: "auto" }
n_estimators=50 #@param {type:"slider", min:50, max:250, step:50}
max_depth=10 #@param {type:"slider", min:2, max:25, step:1}

RF_clf=RandomForestClassifier(n_estimators=n_estimators,
                              max_depth=max_depth,

                              oob_score=True
                              )

%time
RF_clf.fit(X_train, y_train)

y_predict=RF_clf.predict(X_val)
```

n_estimators50

max_depth10

[Скрыть код](#)

CPU times: user 4 µs, sys: 0 ns, total: 4 µs
Wall time: 8.58 µs

```
%time
cross_validation (X_train, y_train,
                  RF_clf, scoring,
                  cv_clf)
```

CPU times: user 3 µs, sys: 1 µs, total: 4 µs
Wall time: 6.44 µs
Ошибка на кросс-валидации

	fit_time	score_time	test_accuracy	test_precision	test_recall	test_f1_score
0	10.021708	0.407511	0.883571	0.882536	0.881982	0.882110
1	10.397774	0.392154	0.885397	0.884494	0.883890	0.884033
2	10.382211	0.368712	0.885476	0.884492	0.884013	0.884152
3	9.942919	0.635144	0.885873	0.885190	0.884537	0.884652
4	9.267237	0.370739	0.881190	0.880378	0.879666	0.879838

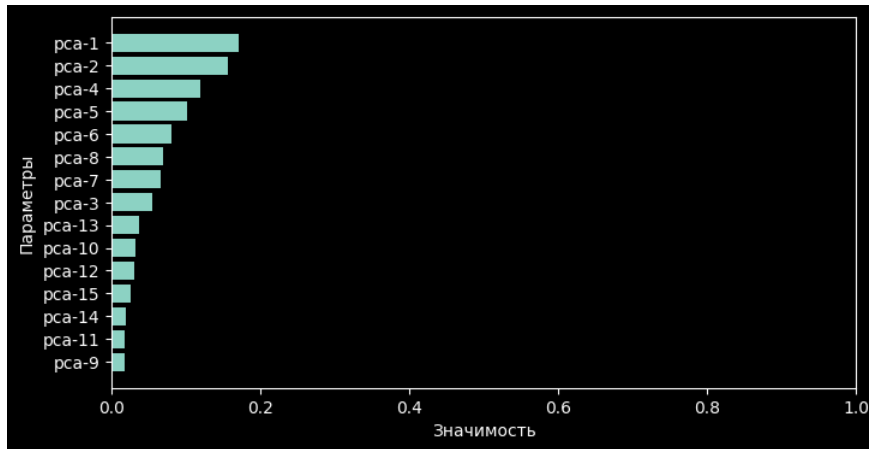
test_accuracy 0.884302
test_precision 0.883418
test_recall 0.882818
test_f1_score 0.882957
dtype: float64

```
print (classification_report(y_val, y_predict, target_names=y_train.categories))
```

	precision	recall	f1-score	support
0	0.95	0.93	0.94	683
1	0.95	0.97	0.96	800
2	0.87	0.86	0.86	674
3	0.83	0.87	0.85	760
4	0.87	0.86	0.86	611
5	0.83	0.87	0.85	658
6	0.92	0.93	0.93	677
7	0.93	0.89	0.91	724
8	0.82	0.81	0.81	693
9	0.85	0.83	0.84	720
accuracy			0.88	7000
macro avg	0.88	0.88	0.88	7000
weighted avg	0.88	0.88	0.88	7000



```
feature_importance_plotter(RF_clf, features_names)
```



Shap

Надо устанавливать библиотеку Shap - но тут стандартно

```
!pip install shap
```

```
Collecting shap
  Downloading shap-0.43.0-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (532 kB)
    532.9/532.9 kB 7.5 MB/s eta 0:00:00
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from shap) (1.23.5)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from shap) (1.11.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from shap) (1.2.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from shap) (1.5.3)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.10/dist-packages (from shap) (4.66.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.10/dist-packages (from shap) (23.2)
Collecting slicer==0.0.7 (from shap)
  Downloading slicer-0.0.7-py3-none-any.whl (14 kB)
Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from shap) (0.56.4)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.10/dist-packages (from shap) (2.2.1)
Requirement already satisfied: llvmlite<0.40,>=0.39.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba->shap) (0.39.1)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from numba->shap) (67.7.2)
Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2023.3.post1)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (3.2.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.1->pandas->shap) (1.16.0)
Installing collected packages: slicer, shap
Successfully installed shap-0.43.0 slicer-0.0.7
```

```
import shap
shap.initjs()
```



Интерфейс работы в shap

```
TreeExplainer
```

потому что лес это много деревьев

```
%time
explainer_rf = shap.TreeExplainer(RF_clf)
max_num = 100
shap_values_rf = explainer_rf(X_val[:max_num])
```

```
CPU times: user 3 µs, sys: 1 µs, total: 4 µs
Wall time: 7.63 µs
```

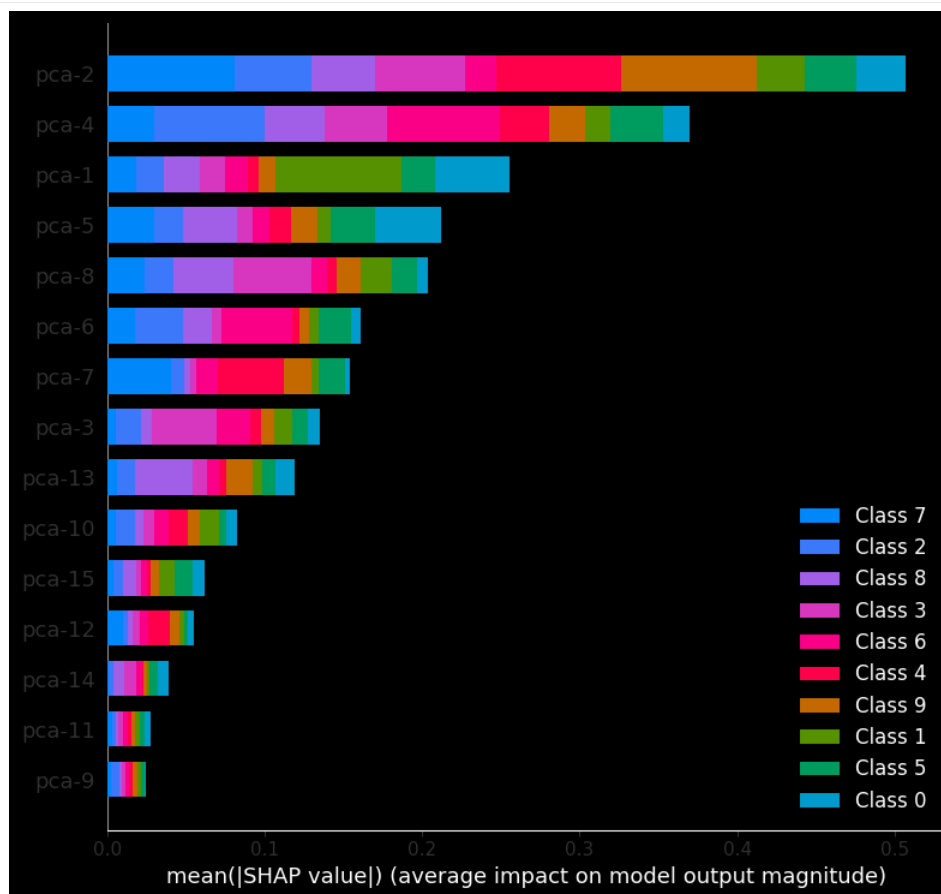
```
shap_values_rf
```

График поинтересней

```
shap.summary_plot(
    [shap_values_rf[:, :, class_ind].values for class_ind in range(shap_values_rf.shape[-1])],
    feature_names=features_names,
```

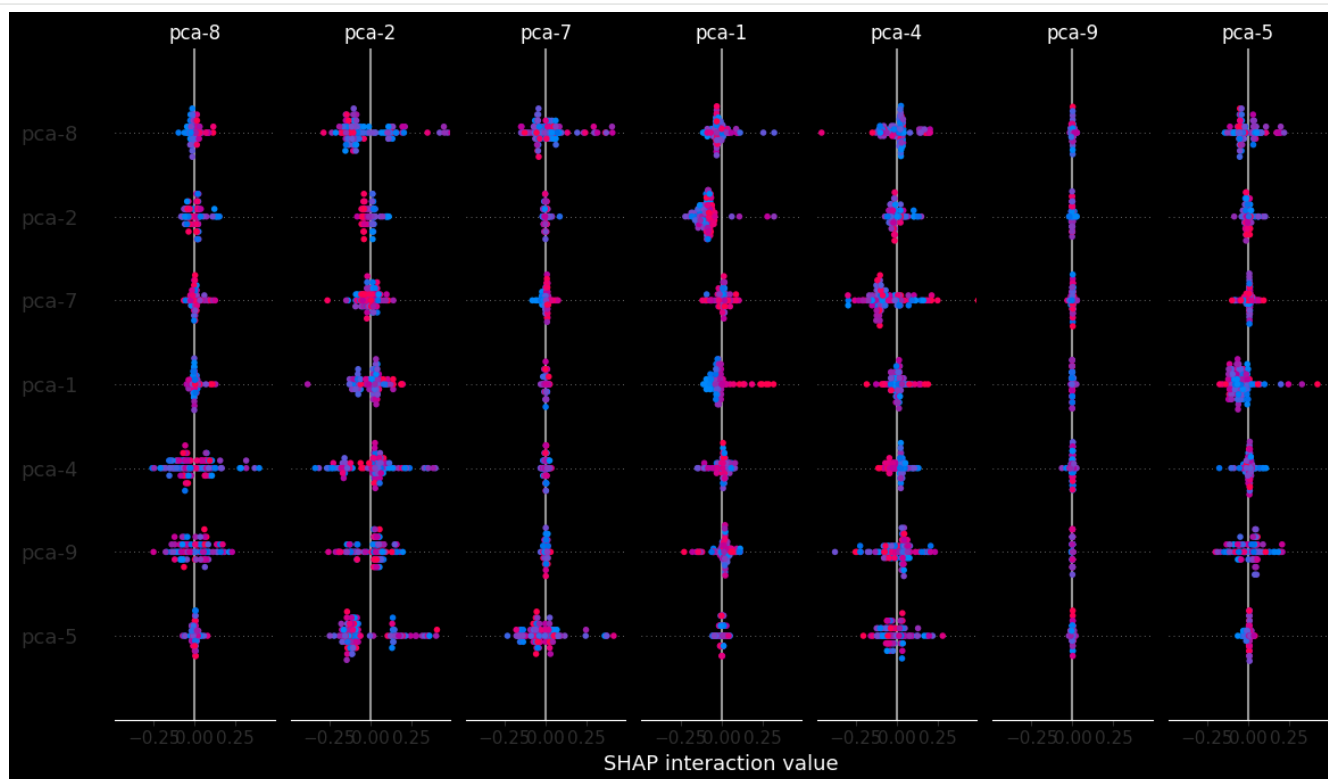


```
plot_type="bar")
```



parirplot от мира [Shap](#)

```
shap.summary_plot(shap_values_rf,  
                  feature_names=features_names,)
```



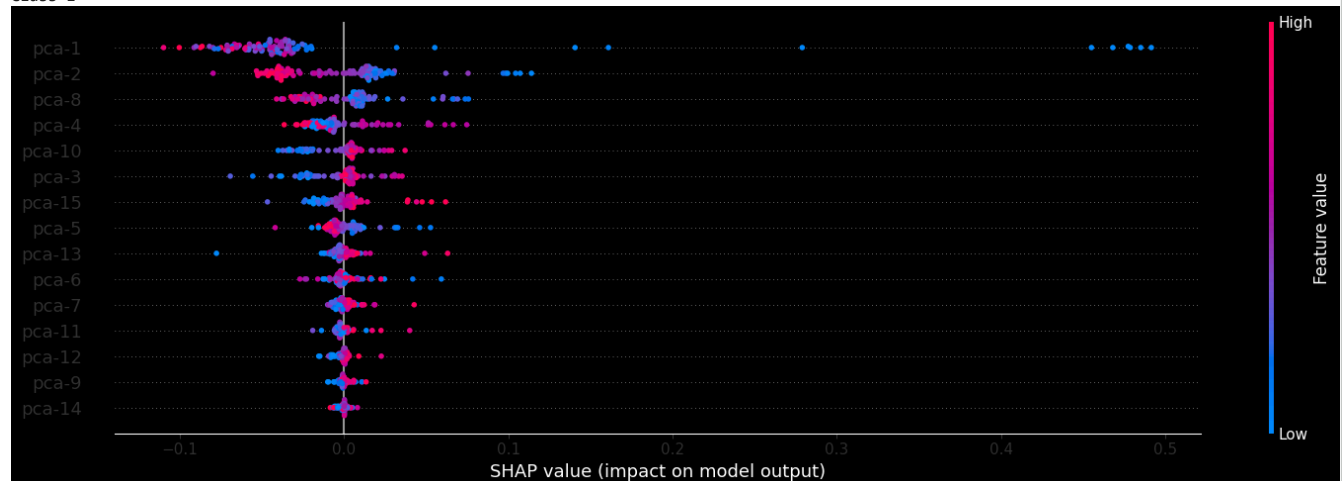
```
# Или на каждый класс по отдельности
for i in range(len(np.unique(y.categories))):
    print(f'Class {i}')
    shap.summary_plot(shap_values_rf[:,i], X_val[:max_num],
                      feature_names=features_names,
                      color_bar=True, plot_size = (15, 5))

    if i == 2:
        break
```

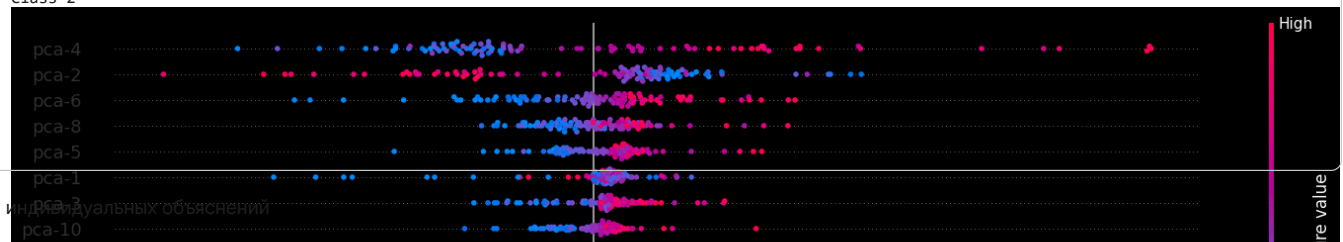
Class 0



Class 1



Class 2



Для индивидуальных объяснений

explainer_rf.expected_value[0]

```
0.09886507936507938
```

shap_values_rf[index,:,0]



```
.base_values =
0.09886507936507938
```

```
.data =
array([ 2.84408721, -1.58277275, -1.28187265, -1.80069433,  3.38617435,
        2.42006342, -1.29547728, -0.40540914, -0.62322127,  0.53469753,
        2.48870252, -1.11629911,  2.52886617, -0.14389802,  0.26521906])
```

index

number



```

#@title { run: "auto" }
index = 0 #@param {type:"slider", min:0, max:100, step:1}
number = 1 #@param {type:"slider", min:0, max:9, step:1}

print(y_val[index])
shap.initjs() # надо добавлять для графиков где необходим JS
shap.plots.force(shap_values_rf[index,:,number],
                 feature_names=features_names,)

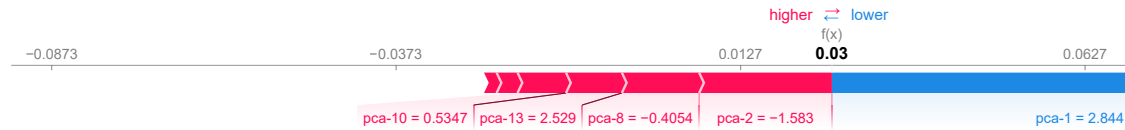
```

0

1

[Скрыть код](#)

8



Еще один индивидуальный график

```

#@title { run: "auto" }
index = 0 #@param {type:"slider", min:0, max:100, step:10}
number = 7 #@param {type:"slider", min:0, max:9, step:1}

print(y_val[index])

shap.plots.waterfall(shap_values_rf[index,:,number],
                    )

```

index

number

0

7

[Скрыть код](#)

8

